

Method overriding

- The overridden method cannot be more restrictive in access modifier.

A subclass must be usable anywhere the parent is expected.

public → must stay *public*

protected → can stay *protected* or become *public*

You **cannot** go from *public* → *protected*

You **cannot** go from *protected* → *private*

Method overriding

- Inheritance is an “is-a” relationship.
- Dog is an Animal

Then anything you can legally do with an **Animal**, you must also be able to do with a **Dog**.

Animal



Dog



Polymorphism

The object created is the **same** in both cases: *new Dog()*

```
Animal a = new Dog();
```

vs

```
Dog a = new Dog();
```

- Reference type: `Animal`
- Object type: `Dog`

This is called **upcasting**.
the compiler only allows what `Animal` promises

Now you can only access:

- Methods defined in `Animal`

- Reference type: `Dog`
- Object type: `Dog`

You can access:

- Methods from `Dog`
- Methods inherited from `Animal`

Overriding

- Everything works again if you **override**

```
class Animal {  
    public void makeSound() {  
        System.out.println("Animal sound");  
    }  
}
```

```
class Dog extends Animal {  
    @Override  
    public void makeSound() {  
        System.out.println("Bark");  
    }  
}
```

```
Animal a = new Dog();  
a.makeSound();
```

Why Would We Ever Do That?

Because **Polymorphism**

```
List<Animal> animals = new ArrayList<>();  
  
animals.add(new Dog());  
animals.add(new Cat());  
animals.add(new Cow());
```

This is **loose coupling** and scalable design

```
for (Animal a : animals) {  
    a.makeSound();  
}
```

CSC1031

OOP

Abstract Classes and Interfaces in Java

Introduction to Abstraction

What is Abstraction?

- Hiding implementation details and exposing only the necessary parts
- Helps in designing flexible and scalable systems

Why is Abstraction important in OOP?

- Promotes code reusability
- Supports loose coupling
- Enhances maintainability

Types of Abstraction in Java

Two ways to achieve abstraction:

- Abstract Classes (partial abstraction)
- Interfaces (full abstraction)

Differences in abstraction level

- Abstract class: 0-100% abstraction
- Interface: 100% abstraction (before Java 8)

Abstract Classes - Definition

What is an Abstract Class?

- A class that cannot be instantiated
- Can have both abstract and concrete methods
- Used for defining a base class with common behavior

```
abstract class Animal {  
    abstract void makeSound();  
    void eat() {  
        System.out.println("This animal eats food.");  
    }  
}
```

Abstract vs. Concrete Methods

Abstract Methods

- A method declared without an implementation (no method body).
- It must be defined inside an abstract class or an interface.
- Subclasses must override it.

```
abstract class Animal {  
    abstract void makeSound(); // No implementation  
}
```

Abstract vs. Concrete Methods

Concrete Methods

- A method that has a body (implementation).
- It can exist in regular and abstract classes.
- Can be overridden, but it's not mandatory.

```
class Dog {  
    void bark() {  
        System.out.println("Barking...");  
    }  
}
```

Abstract vs. Concrete Methods

Feature	Abstract Method	Concrete Method
Implementation	No method body (just declaration)	Has method body
Location	Inside an abstract class or interface	In any class (including abstract ones)
Override Requirement	Must be implemented in a subclass	Optional override
Usage	Defines behavior that subclasses must implement	Provides ready-to-use behavior

Abstract vs. Concrete Methods

```
abstract class Animal {  
    abstract void makeSound(); // Abstract method  
  
    void eat() { // Concrete method  
        System.out.println("Eating food...");  
    }  
}  
  
class Dog extends Animal {  
    void makeSound() {  
        System.out.println("Bark!");  
    }  
}
```

Abstract Classes - Key Features

Can have constructors

Can have concrete (non-abstract) methods

Can define instance variables

Can have final, static, and non-static methods

Must be extended by a subclass

```
class Dog extends Animal {  
    void makeSound() {  
        System.out.println("Bark!");  
    }  
}
```

Interfaces - Definition

What is an Interface?

- A **contract** that a class must follow
- Can only contain abstract methods (before Java 8)
- Cannot have instance variables
- Supports multiple inheritance

```
interface Animal {  
    void makeSound();  
}
```

Interfaces - Key Features

- All methods are public and abstract by default
- Variables are public, static, and final by default
- A class implements an interface using **implements** keyword
- Supports multiple inheritance

```
interface Animal {  
    void makeSound();  
}
```

```
class Dog implements Animal {  
    public void makeSound() {  
        System.out.println("Bark!");  
    }  
}
```

Differences Between Abstract Classes and Interfaces

Feature	Abstract Class	Interface
Method Type	Abstract + Concrete	Only Abstract (Java < 8)
Variables	Instance + Static	Only Static + Final
Constructors	Yes	No
Inheritance	Single	Multiple
Access Modifiers	Any	Only public

Choose abstract classes for base implementations, and interfaces for **behavior contracts**.

Contract or Behavior Contract

What is a Behavior Contract?

- A set of rules that a class must follow
- Defined using interfaces
- Ensures consistency across multiple classes implementing the same interface.
- Helps in loosely coupled and scalable design.

How Does a Contract Work?

An interface defines a contract (**set of methods**).

Any class that implements the interface **must** provide implementations for all methods.

Ensures that all implementing classes follow the same behavior.

```
interface PaymentProcessor {  
    void processPayment(double  
amount);  
}
```

- The interface acts as a **contract** that any payment processor **must** implement.
- This ensures that all payment methods (e.g., PayPal, CreditCard, Bitcoin) follow the **same structure**.

```
class PayPalPayment implements PaymentProcessor {  
    public void processPayment(double amount) {  
        System.out.println("Processing PayPal payment of $" + amount);  
    }  
}  
  
class CreditCardPayment implements PaymentProcessor {  
    public void processPayment(double amount) {  
        System.out.println("Processing Credit Card payment of $" + amount);  
    }  
}
```

Interfaces After Java 8

New features introduced in Java 8+

- Default methods
- Static methods

```
interface Animal {  
    default void sleep() {  
        System.out.println("Sleeping...");  
    }  
}
```

Default Methods

What Are Default Methods?

- Introduced in Java 8 to allow **method implementation inside interfaces**.
- Provide a default behavior so that implementing classes **aren't forced to override** them.
- Useful for backward compatibility (adding methods to interfaces without breaking existing code).

```
interface Animal {  
    default void sleep() {  
        System.out.println("Sleeping...");  
    }  
}
```

Default Methods

```
class Dog implements Animal {  
    // No need to override sleep() unless we want a custom implementation  
}  
  
class Cat implements Animal {  
    public void sleep() { // Overriding the default method  
        System.out.println("Cat sleeps quietly...");  
    }  
}
```

Default Methods

When **NOT** to Use Default Methods?

When the method is not logically shared across all implementations.

If the method needs to access instance variables, which interfaces do not support.

When complex logic is needed—better to use abstract classes instead.

When to Use Abstract Classes vs Interfaces

Use an **Abstract** Class when:

- You need to share common fields or methods
- You want to provide partial implementation

Use an **Interface** when:

- You want to define a contract for multiple classes
- You need multiple inheritance capabilities

Common Mistakes and Best Practices

Common Mistakes:

- Trying to instantiate an abstract class
- Forgetting to implement all interface methods
- Mixing up extends and implements

Best Practices:

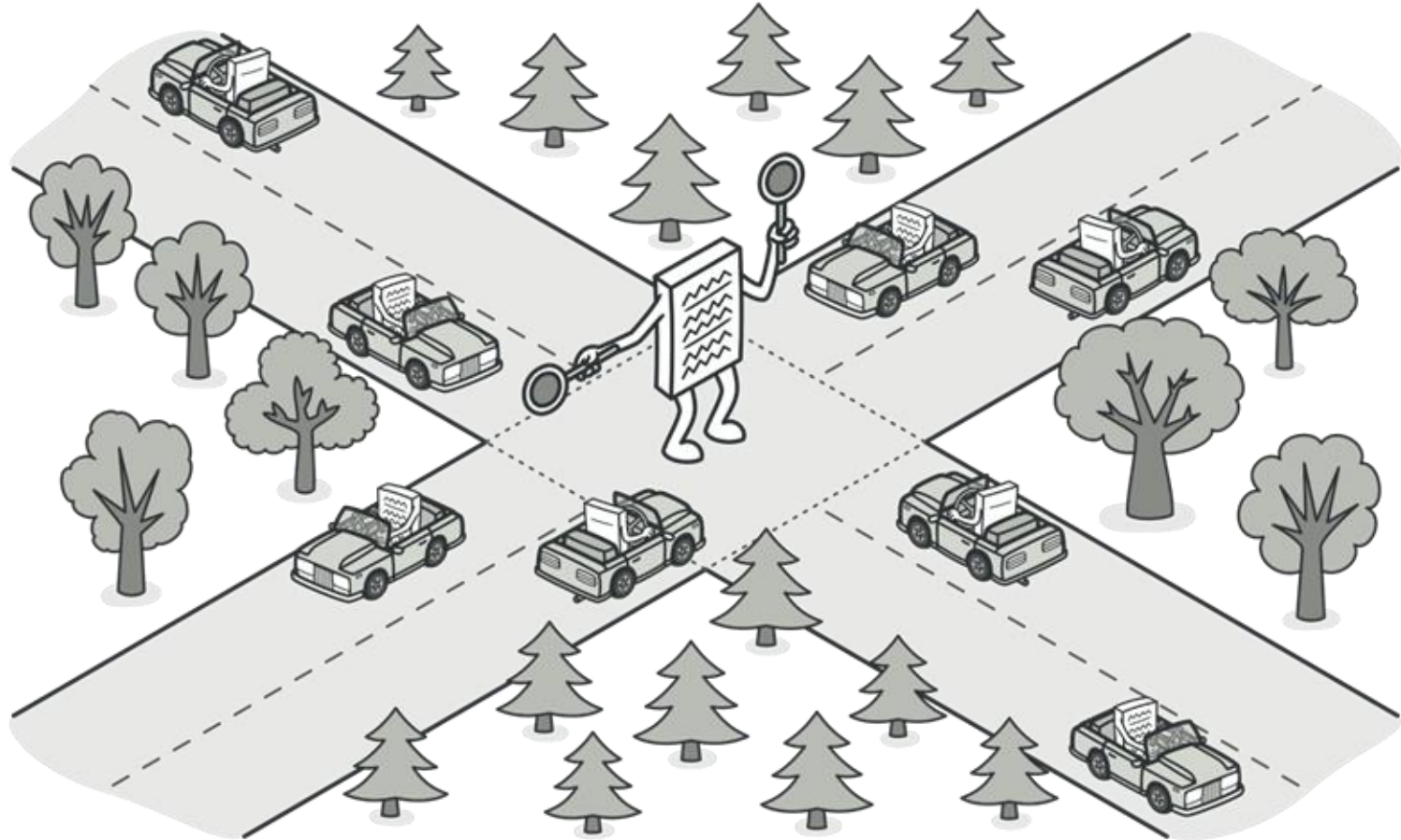
- Use meaningful names for abstract classes and interfaces
- Keep interfaces focused on a single responsibility
- Prefer interfaces when designing APIs

Design Patterns

Mediator

Design Patterns

Mediator



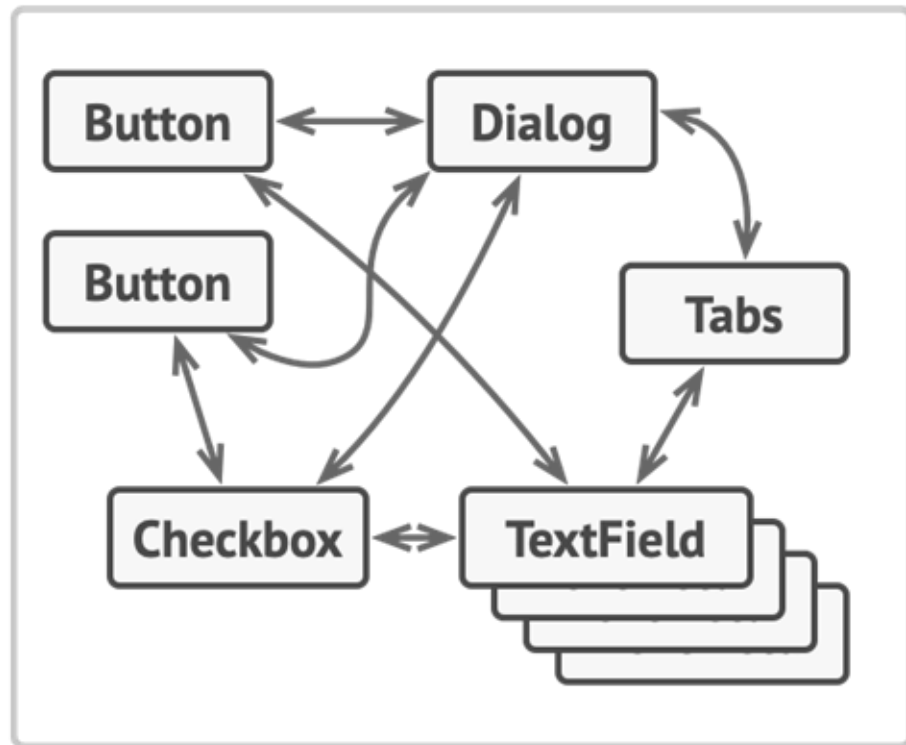
Mediator

Mediator is a **behavioral** design pattern that lets you reduce chaotic dependencies between objects.

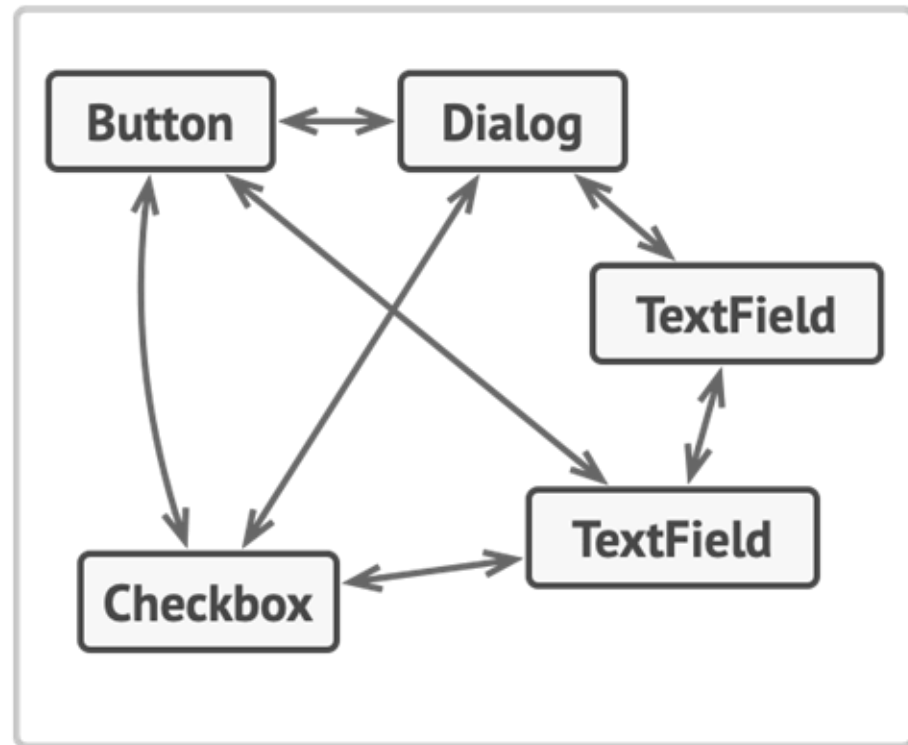
The pattern restricts direct communications between the objects and forces them to collaborate only via a mediator object.

Problem

Profile Dialog



Login Dialog



Problem

Elements may interact with others. For instance, selecting the “I have a dog” checkbox may reveal a hidden text field for entering the dog’s name. Another example is the submit button that has to validate values of all fields before saving the data.

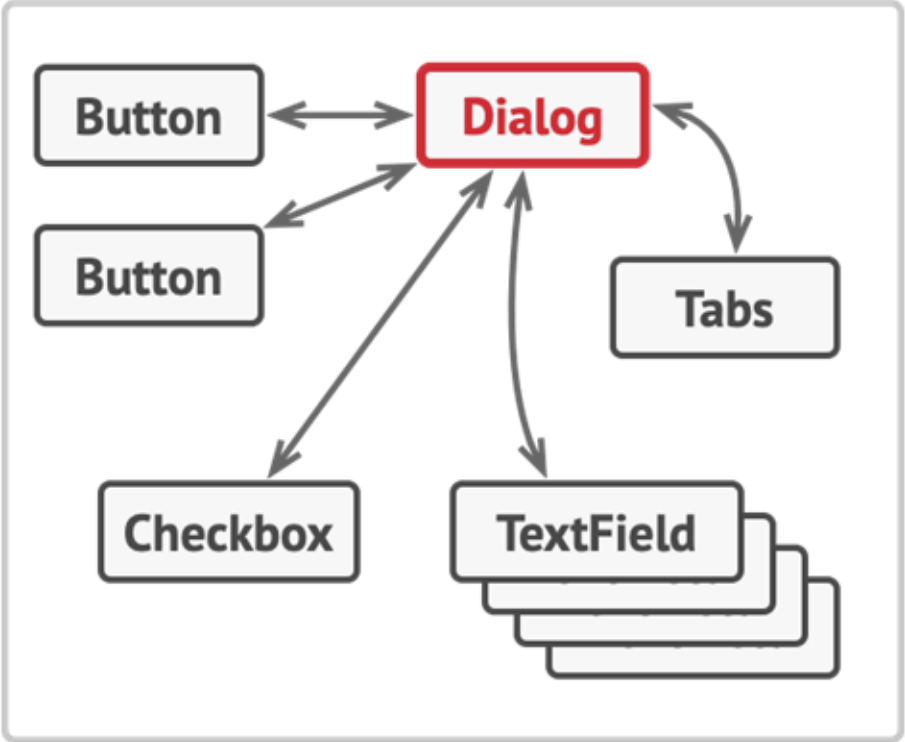


Solution

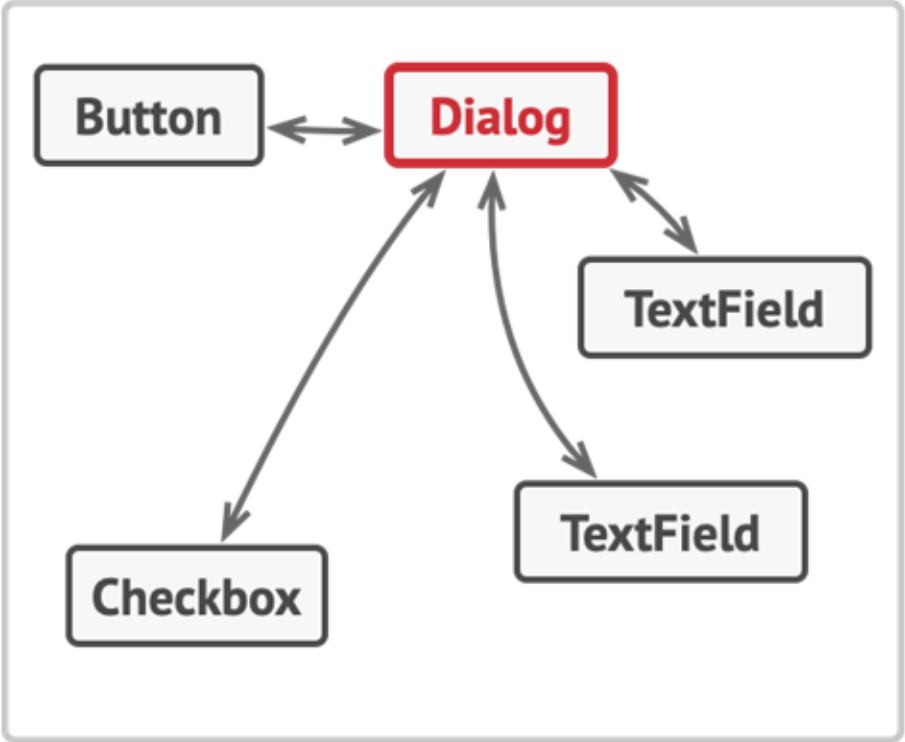
- The Mediator pattern suggests that you should **cease all direct communication between the components** which you want to make independent of each other.
- These components must collaborate indirectly, by **calling a special mediator object** that redirects the calls to appropriate components.
- The **components depend only on a single mediator class** instead of being coupled to dozens of their colleagues.

Solution

Profile Dialog



Login Dialog



Explanation

Submit button:

- Previously, each time a user clicked the button, it had to validate the values of all individual form elements
- With mediator now its single job is to notify the dialog about the click
- Upon receiving this notification, the dialog itself performs the validations or passes the task to the individual elements
- Instead of being tied to a dozen form elements, the button is only dependent on the dialog class.

Explanation

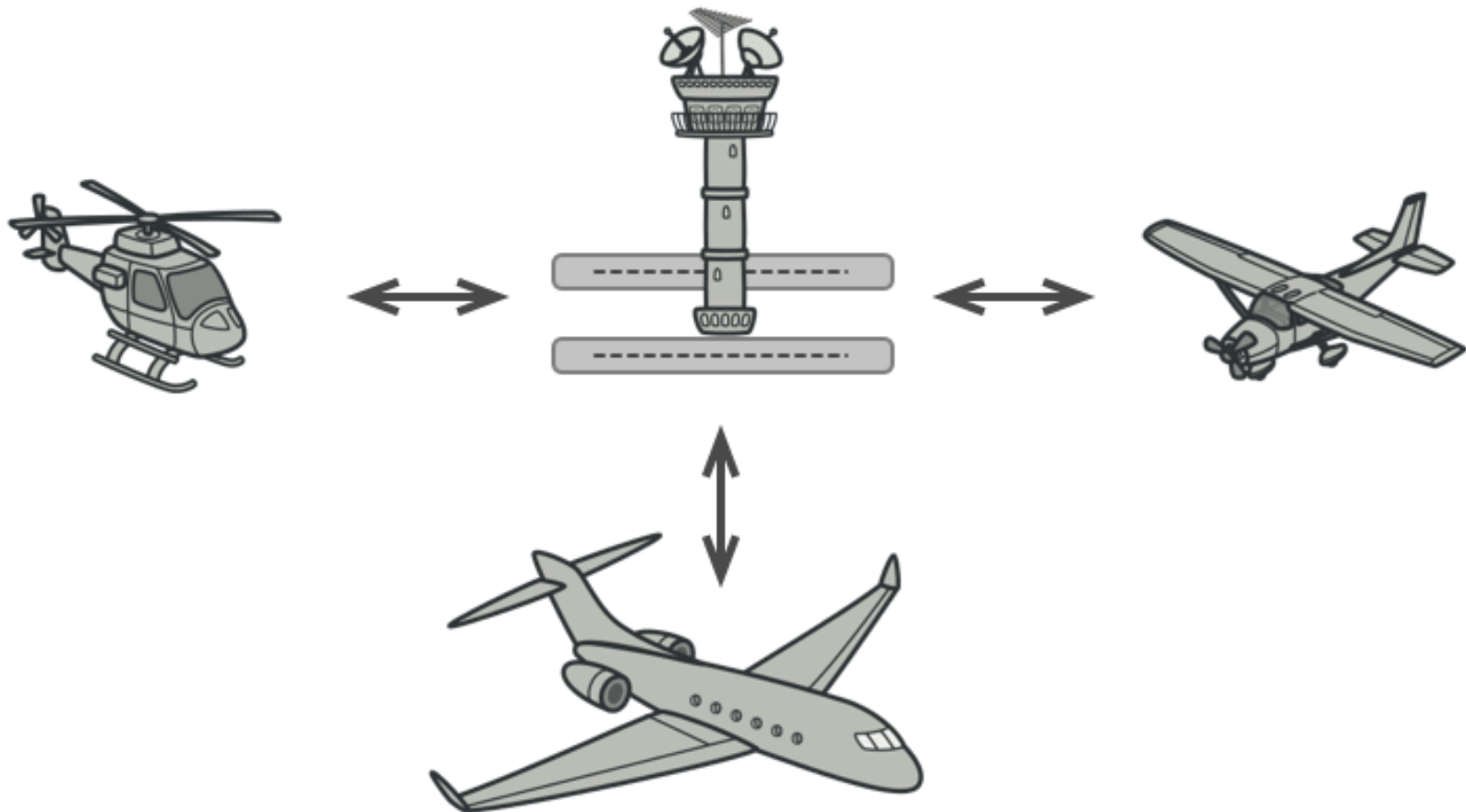
You can make the dependency even looser by extracting the common interface for all types of dialogs.

The interface would declare the notification method (contract) which all form elements can use to notify the dialog about events happening to those elements.

Submit button should now be able to work with any dialog that implements that interface.

The Mediator pattern lets you encapsulate a complex web of relations between various objects inside a single mediator object.

The fewer dependencies a class has, the easier it becomes to modify, extend or reuse that class.

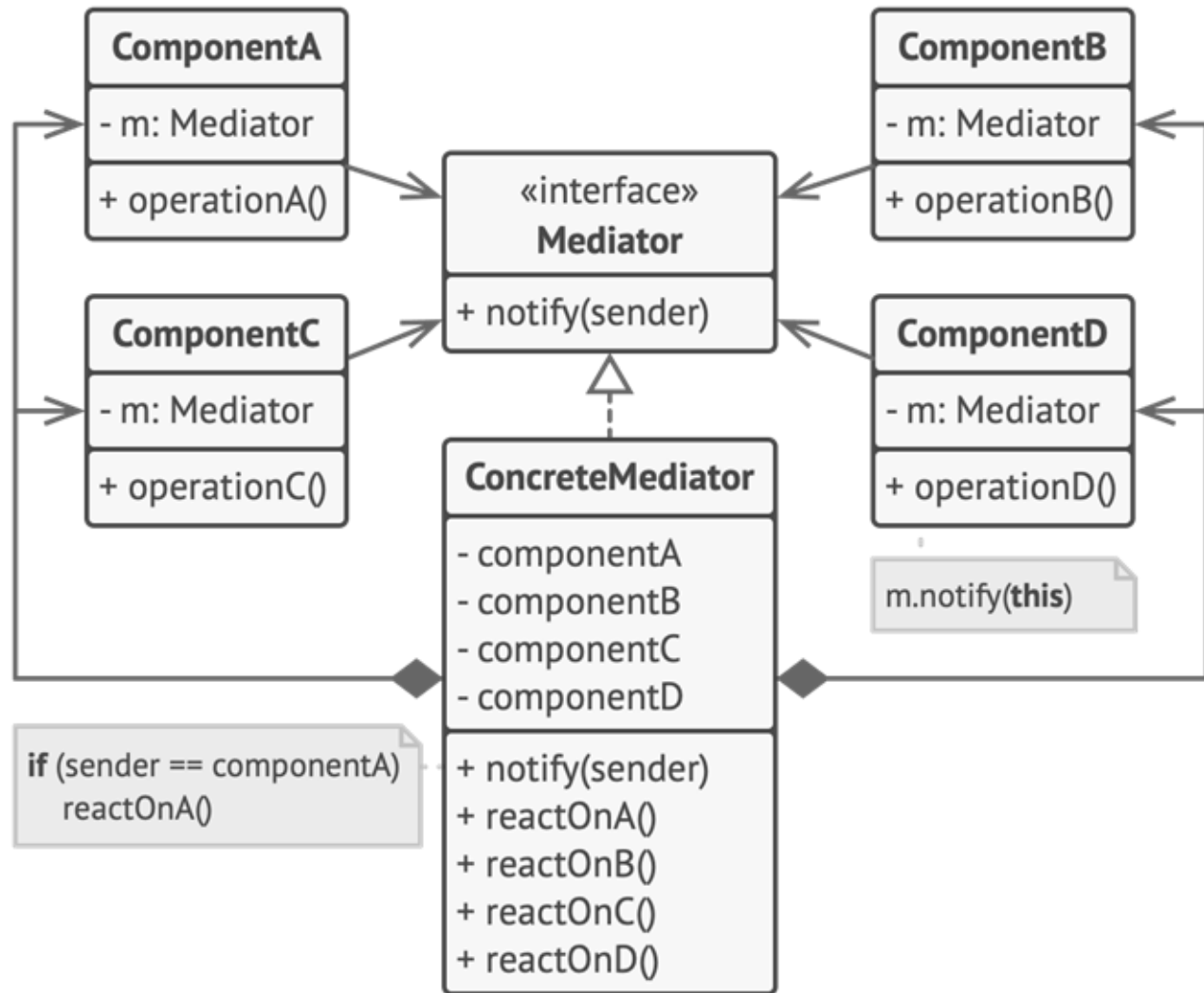


Structure

Components are various classes that contain some business logic.

Each component has a reference to a mediator, declared with the type of the mediator interface.

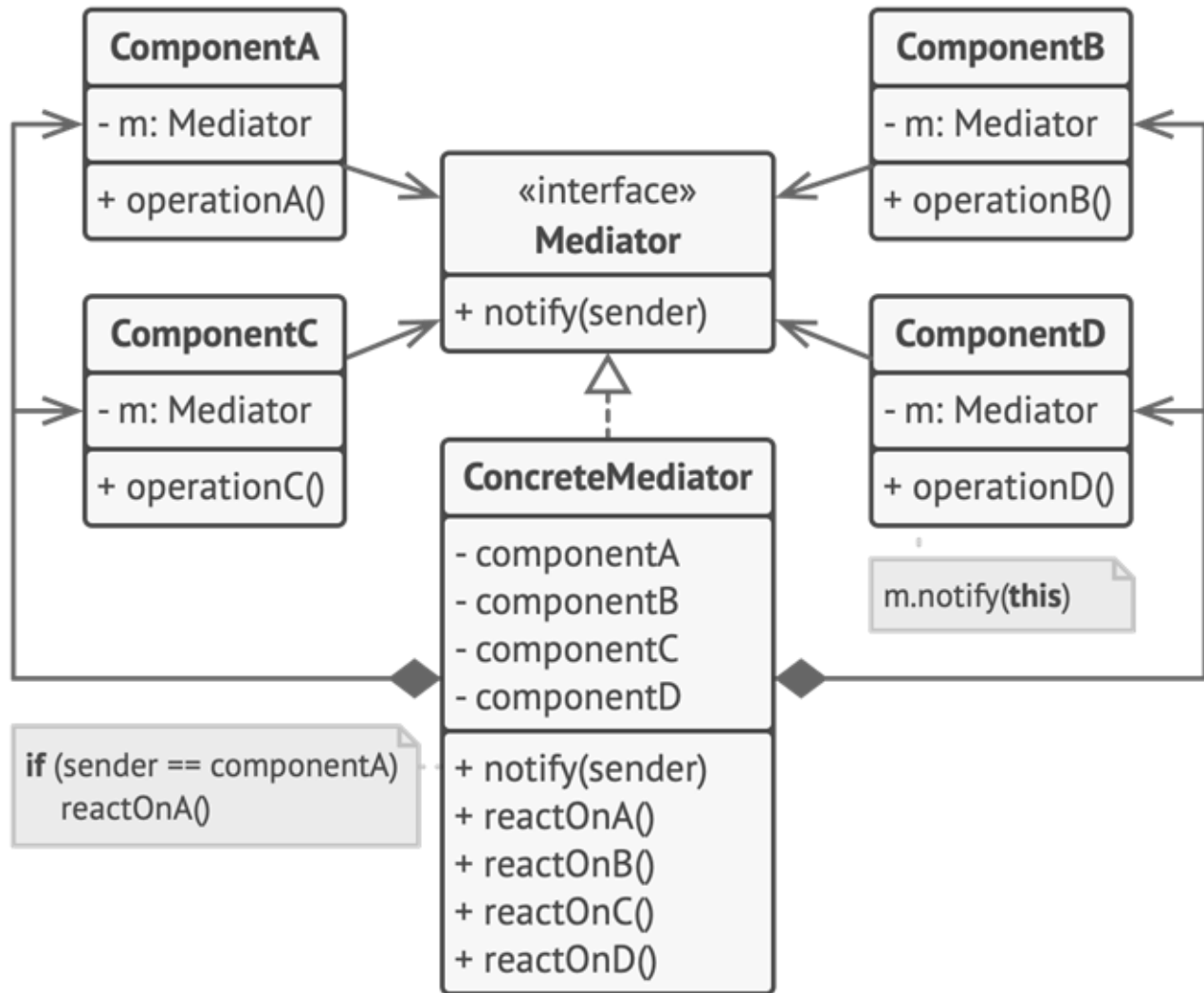
The component isn't aware of the actual class of the mediator, so you can reuse the component in other programs by linking it to a **different mediator**.



Structure

The **Mediator interface** declares methods of communication with components, which usually include just a single notification method.

Components may pass any context as arguments of this method, including their own objects, but only in such a way that **no coupling occurs between a receiving component and the sender's class**.

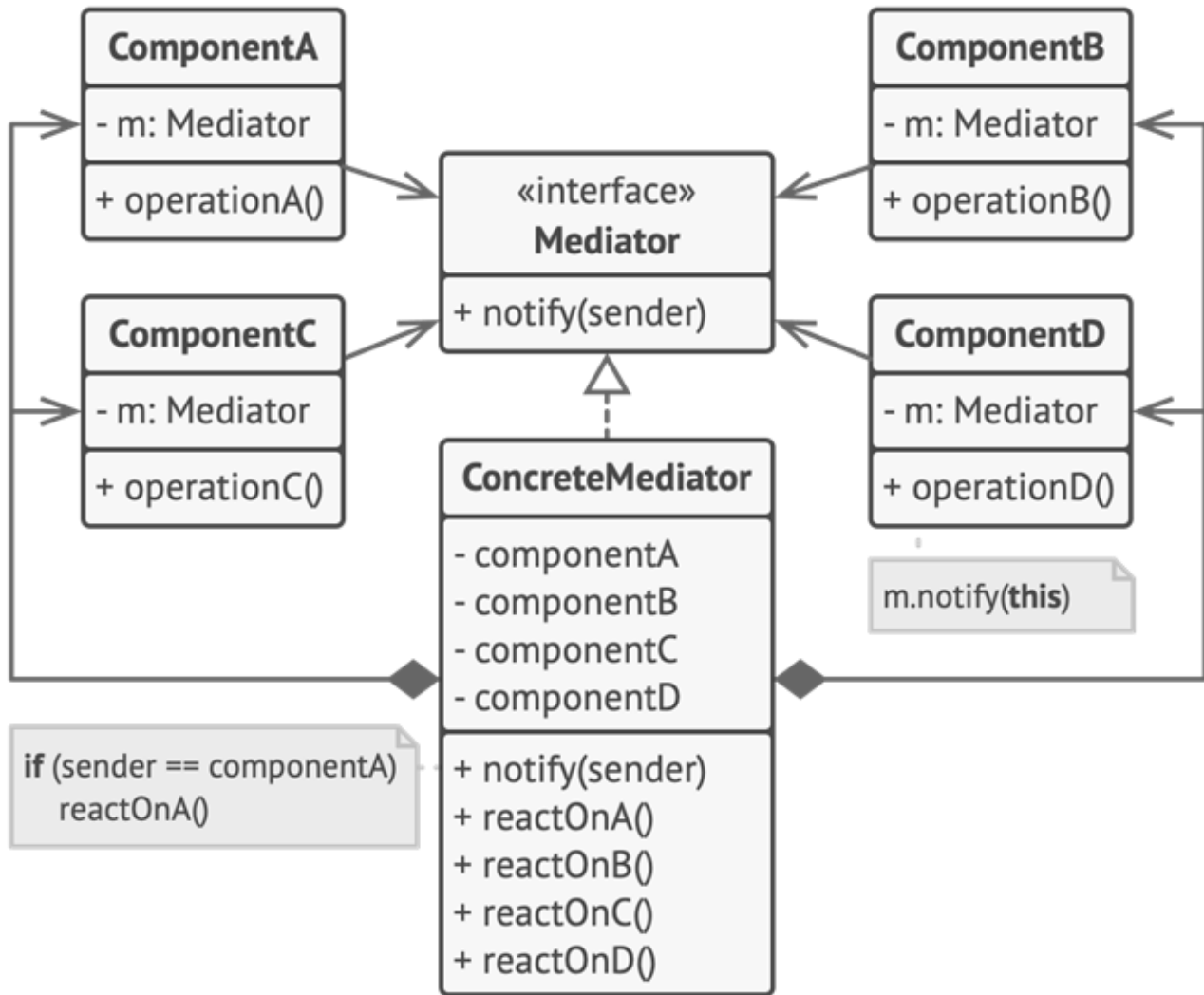


Structure

Concrete Mediators

encapsulate relations between various components.

Concrete mediators often keep **references to all components** they manage and sometimes even manage their lifecycle.



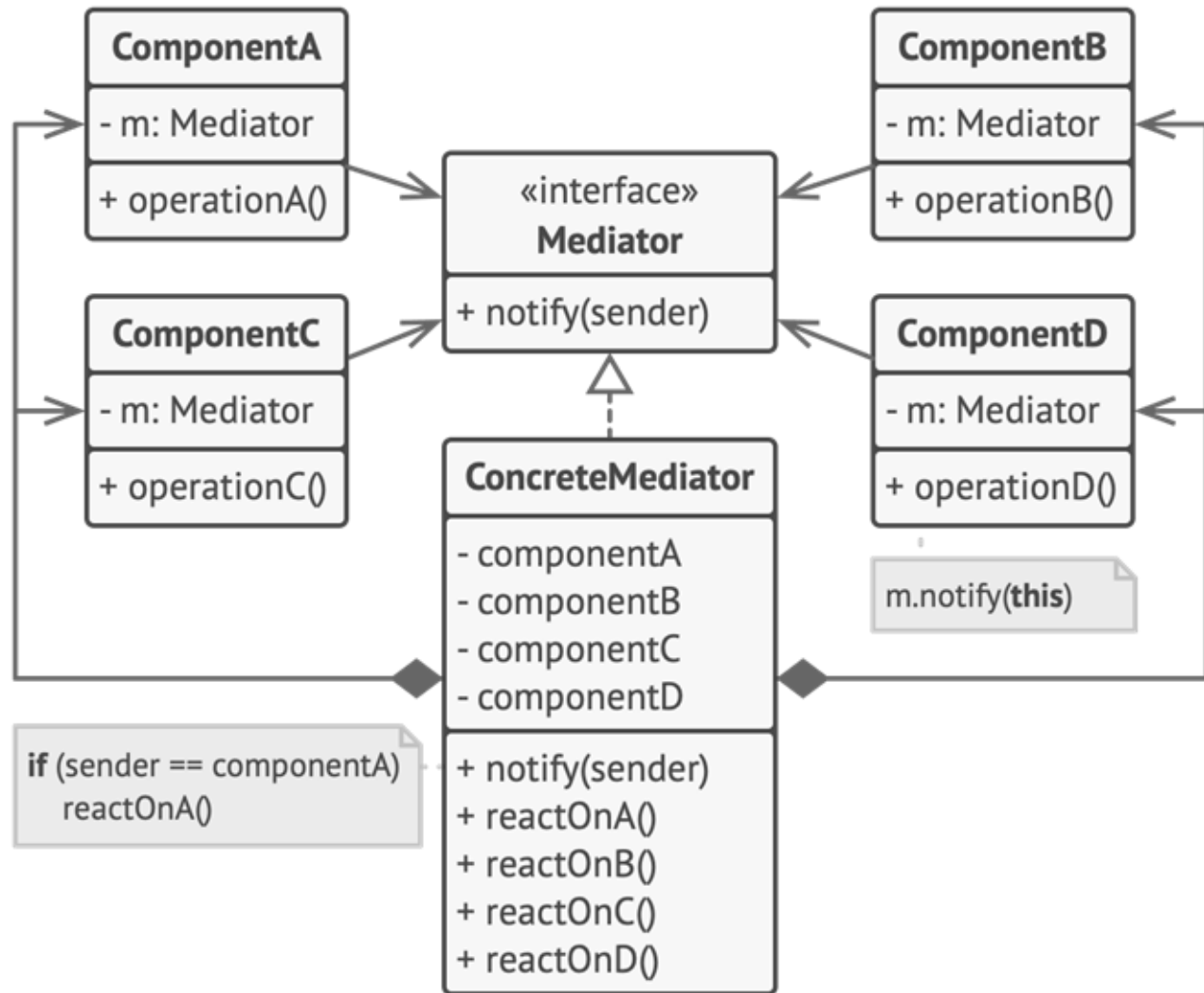
Structure

Components must not be aware of other components.

If something important happens within or to a component, it must only notify the mediator.

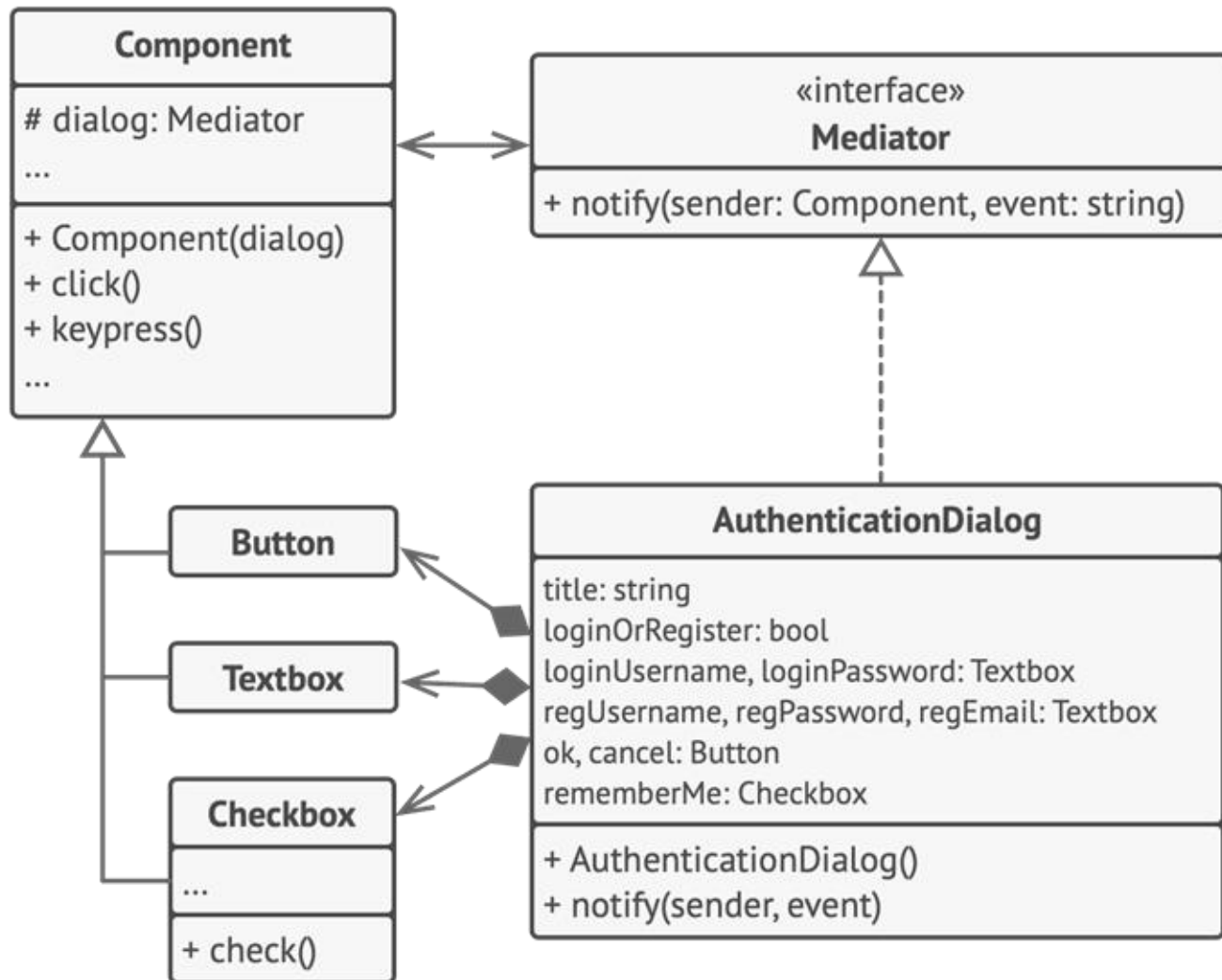
When the mediator receives the notification, it can easily identify the sender, which might be just enough to decide what component should be triggered in return.

From a component's perspective, it all looks like a total black box. The sender doesn't know who'll end up handling its request, and the receiver doesn't know who sent the request in the first place.



Structure

A triggered element doesn't communicate with other elements directly, even if it looks like it's supposed to. Instead, the element only needs to let its mediator know about the event, passing any contextual info along with that notification.



Applicability

Use the Mediator pattern when it's hard to change some of the classes because they are tightly coupled to a bunch of other classes.

The pattern lets you extract all the relationships between classes into a separate class, isolating any changes to a specific component from the rest of the components.

Applicability

Use the pattern when you can't reuse a component in a different program because it's too dependent on other components.

After you apply the Mediator, individual components become unaware of the other components. They could still communicate with each other, albeit indirectly, through a mediator object. To reuse a component in a different app, you need to provide it with a new mediator class.

Pros

Single Responsibility Principle. You can extract the communications between various components into a single place, making it easier to comprehend and maintain.

Open/Closed Principle. You can introduce new mediators without having to change the actual components.

You can **reduce coupling between various components** of a program.

You can **reuse** individual components more easily.

and Cons

Over time a mediator can evolve into a God Object

